

ФГБОУ ВО «Пензенский государственный университет»
Политехнический институт
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №5
По дисциплине «Логика и основы
алгоритмизации в инженерных задачах»
На тему «Определение характеристик графов»

Выполнили: ст. гр. 25ВВВ1
Разживалов Н.А.
Цеплов Е.В.
Проверили: к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Определение характеристик графов

Цель работы

Научиться реализовывать графы и производить операции над ними

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Определите размер графа G , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Задание 2

1. Постройте для графа G матрицу инцидентности.
2. Определите размер графа G , используя матрицу инцидентности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Ход работы

В main реализовали бесконечный цикл while с конструкцией switch внутри, для возможности выбрать нужное действие.

```
int main(){
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));
    int** g = NULL;
    int n, r;
    int choice;
    while (1) {
        printf("\nвыберите действие и введите его номер:\n 1 - создать граф\n 2 - задать случайные связи вершин графа\n 3 - вывести граф\n 4 - посчитать количество рёбер\n 5 - тип вершины\n 6 - очистить граф\n 0 - выход\n");
        scanf_s("%d", &choice);
        switch (choice) {
            case 1: g = createGraph(&n); break;
            case 2: randGraph(g, n); break;
            case 3: printGraph(g, n); break;
            case 4: r = rcount(g, n); printf("%d", r); break;
            case 5: typevertex(g, n); break;
            case 6: freeGraph(g, n); return 0;
            default: break;
        }
    }
}
```

Реализовали функцию создания графа при помощи динамического выделения памяти.

```
int** createGraph(int* n) {
    int** g;
    int i;
    printf("Введите количество вершин графа ");
    do {
        scanf_s("%d", n);
        if ((*n) <= 0) {
            printf("введите положительное ненулевое число! ");
        }
    } while ((*n) <= 0);

    g = (int**)malloc((*n) * sizeof(int*));

    for (i = 0; i < (*n); i++){
        g[i] = (int*)malloc((*n) * sizeof(int));
    }

    return g;
}
```

Реализовали заполнение массива нулями (связи вершин нет) и единицами (связь вершин есть).

```
void randGraph(int** g, int n) {
    if (g == NULL) {
        printf("Граф ещё не был создан\n");
        return;
    }

    for (int i = 0; i < n; i++) {
        g[i][i] = 0;
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            g[i][j] = rand() % 2;
            g[j][i] = g[i][j];
        }
    }

    printGraph(g, n);
}
```

Реализовали функцию нахождения размера графа (количества рёбер).

```
int rcount(int** g, int n){
    if (g == NULL) {
        printf("Граф ещё не был создан\n");
        return 0;
    }

    int r = 0;
    for (int i = 0; i < n; i++){
        for (int j = i + 1; j < n; j++){
            if (g[i][j] == 1) {
                r++;
            }
        }
    }

    return r;
}
```

Реализовали функцию определения типа вершин графа через подсчёт количества рёбер.

```
void typevertex(int** g, int n) {
    if (g == NULL) {
        printf("Граф ещё не создан\n");
        return;
    }
    int t = 0;
    for (int i = 0; i < n; i++) {
        t = 0;
        for (int j = 0; j < n; j++) {
            t += g[i][j];
        }
        if (t == n - 1) {
            printf("Вершина %d - доминирующая\n", i+1);
        }
        if (t == 1) {
            printf("Вершина %d - конечная\n", i+1);
        }
        if (t == 0) {
            printf("Вершина %d - изолированная\n", i+1);
        }
    }
}
```

Реализовали матрицу инцидентности. Для этого оставили строки вершинами, а столбцы заменили на рёбра.

```
int** createInc(int** g, int n, int m) {
    if (g == NULL) {
        printf("Граф ещё не создан\n");
        *m = 0;
        return NULL;
    }

    *m = rcount(g, n);

    int** inc = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        inc[i] = (int*)calloc(*m, sizeof(int));
    }

    int edgeIndex = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (g[i][j] == 1) {
                inc[i][edgeIndex] = 1;
                inc[j][edgeIndex] = 1;
                edgeIndex++;
            }
        }
    }

    return inc;
}
```

Внесли изменения в функции, чтобы они основывались уже на инцидентной матрице.

```
void typevertexInc(int** inc, int n, int m) {
    if (inc == NULL) {
        printf("Матрица инцидентности ещё не была создана\n");
        return;
    }
    for (int i = 0; i < n; i++) {
        int t = 0;
        for (int j = 0; j < m; j++) {
            t += inc[i][j];
        }
        if (t == 0) {
            printf("Вершина %d - изолированная\n", i + 1);
        }
        if (t == 1) {
            printf("Вершина %d - конечная\n", i + 1);
        }
        if (t == n - 1 && n > 1) {
            printf("Вершина %d - доминирующая\n", i + 1, t);
        }
    }
}
```

Псевдокод

Задание 1

ФУНКЦИЯ randGraph

ДЛЯ i ОТ 0 ДО $n - 1$ ДЕЛАТЬ

$g[i][i] = 0$

КОНЕЦ ПОКА

ДЛЯ i ОТ 0 ДО $n - 1$ ДЕЛАТЬ

ДЛЯ j ОТ $i + 1$ ДО $n - 1$ ДЕЛАТЬ

$g[i][j] = \text{СЛУЧАЙНОЕ}(0 \text{ ИЛИ } 1)$

$g[j][i] = g[i][j]$

КОНЕЦ ПОКА

КОНЕЦ ПОКА

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ rcount

$r = 0$

ДЛЯ i ОТ 0 ДО $n - 1$ ДЕЛАТЬ

ДЛЯ j ОТ $i + 1$ ДО $n - 1$ ДЕЛАТЬ

ЕСЛИ $g[i][j] = 1$ ТО

$r = r + 1$

КОНЕЦ ЕСЛИ

КОНЕЦ ПОКА

КОНЕЦ ПОКА

ВОЗВРАТ r

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ typevertex

ДЛЯ i ОТ 0 ДО $n - 1$ ДЕЛАТЬ

$t = 0$

ДЛЯ j ОТ 0 ДО $n - 1$ ДЕЛАТЬ

$t = t + g[i][j]$

КОНЕЦ ПОКА

ЕСЛИ $t = 0$ ТО ВЫВЕСТИ "Изолированная" КОНЕЦ ЕСЛИ

ЕСЛИ $t = 1$ ТО ВЫВЕСТИ "Концевая" КОНЕЦ ЕСЛИ
ЕСЛИ $t = n - 1$ ТО ВЫВЕСТИ "Доминирующая" КОНЕЦ ЕСЛИ
КОНЕЦ ПОКА
КОНЕЦ ФУНКЦИИ

Задание 2

ФУНКЦИЯ createInc

$m = \text{rcount}(g, n)$

СОЗДАТЬ матрицу inc размером $n \times m$, заполненную 0

$r = 0$

ДЛЯ i ОТ 0 ДО $n - 1$ ДЕЛАТЬ

 ДЛЯ j ОТ $i + 1$ ДО $n - 1$ ДЕЛАТЬ

 ЕСЛИ $g[i][j] = 1$ ТО

$\text{inc}[i][r] = 1$

$\text{inc}[j][r] = 1$

$r = r + 1$

 КОНЕЦ ЕСЛИ

 КОНЕЦ ПОКА

КОНЕЦ ПОКА

ВОЗВРАТ inc

КОНЕЦ ФУНКЦИИ

ФУНКЦИЯ typevertexInc

ДЛЯ i ОТ 0 ДО $n - 1$ ДЕЛАТЬ

$t = 0$

 ДЛЯ j ОТ 0 ДО $m - 1$ ДЕЛАТЬ

$t = t + \text{inc}[i][j]$

 КОНЕЦ ПОКА

 ЕСЛИ $t = 0$ ТО ВЫВЕСТИ "Изолированная" КОНЕЦ ЕСЛИ

 ЕСЛИ $t = 1$ ТО ВЫВЕСТИ "Концевая" КОНЕЦ ЕСЛИ

 ЕСЛИ $t = n - 1$ ТО ВЫВЕСТИ "Доминирующая" КОНЕЦ ЕСЛИ

КОНЕЦ ПОКА

КОНЕЦ ФУНКЦИИ

Результаты работы программы

```
Граф:
0 0 1 0 0
0 0 0 0 0
1 0 0 1 0
0 0 1 0 1
0 0 0 1 0

выберите действие и введите его номер:
1 - создать граф
2 - задать случайные связи вершин графа
3 - вывести граф
4 - определить размер графа(количество рёбер)
5 - определить типы вершин
4
3
выберите действие и введите его номер:
1 - создать граф
2 - задать случайные связи вершин графа
3 - вывести граф
4 - определить размер графа(количество рёбер)
5 - определить типы вершин
5
Вершина 1 - концевая
Вершина 2 - изолированная
Вершина 5 - концевая
```

```
Граф:
0 0 1 0 1
0 0 1 0 0
1 1 0 1 1
0 0 1 0 1
1 0 1 1 0
Матрица инцидентности:
1 1 0 0 0 0
0 0 1 0 0 0
1 0 1 1 1 0
0 0 0 1 0 1
0 1 0 0 1 1

выберите действие и введите его номер:
1 - создать граф
2 - задать случайные связи вершин графа
3 - вывести граф
4 - определить размер графа(количество рёбер)
5 - определить типы вершин
5
Вершина 2 - концевая
Вершина 3 - доминирующая
```

Вывод

Научились реализовывать графы и производить операции над ними.